

DeepChrInteract-v2: A Modern PyTorch Reimplementation of Deep Learning Methods for Enhancer–Promoter Interaction Prediction

Reproduction, Extension, and Benchmarking with LLM-Based Encoders

May 2026

Abstract

Predicting enhancer–promoter interactions (EPI) from raw DNA sequence is a fundamental task in computational genomics, enabling the systematic study of gene regulation without relying on expensive chromatin capture experiments. We present **DeepChrInteract-v2**, a complete reimplementation and substantial extension of the original DeepChrInteract toolkit [1], which was built on deprecated Keras 1.x/TensorFlow 2.x APIs and has become non-functional due to breaking library changes. The new codebase is built entirely in PyTorch 2.x and introduces four major improvements: (i) a direct in-memory one-hot encoding pipeline that eliminates the original PNG-based intermediate representation; (ii) two new sequence modelling backbones—Bidirectional LSTM (BiLSTM) and Transformer encoder—applied to EPI prediction; (iii) integration of pre-trained DNA large language model (LLM) encoders, including DNABERT [2], DNABERT-2 [3], Nucleotide Transformer [4], and HyenaDNA [5], as frozen or fine-tuned feature extractors; and (iv) a flexible multi-path fusion module that supports concatenation, subtraction, element-wise multiplication, and bilinear interaction between enhancer and promoter representations. A systematic ablation study across eight model configurations evaluates the contribution of each component under a unified evaluation protocol (AUROC, AUPRC, F1), reproducing and substantially extending results from the original work.

Contents

1	Introduction	4
2	Related Work	5
2.1	Chromatin Interaction Prediction	5
2.2	Sequence-Based EPI Prediction	5
2.3	CNN and ResNet for Regulatory Genomics	5
2.4	RNN and LSTM Variants	6
2.5	Transformer Variants for Long Sequences	6
2.6	Linear Recurrence and State Space Models	6
2.7	Self-Supervised Pre-training: MAE	6
2.8	DNA Foundation Models	7
2.9	Multi-Path Interaction Modelling	7
3	Data Format and Encoding Strategies	7
3.1	Data Format	7
3.2	Encoding Strategy A: One-Hot	7
3.3	Encoding Strategy B: k -mer Tokenisation	8
3.4	Encoding Strategy C: Pre-Trained DNA LLM	8
3.5	Data Pipeline Implementation	8
4	Methods	9
4.1	Group A: Classical CNN (Reproduction)	9
4.1.1	M1: CNN Single-Branch	9
4.1.2	M2: CNN Dual-Branch	10
4.1.3	M3: k -mer Embedding + CNN	10
4.2	Group B: RNN / LSTM Family	10
4.2.1	M4: Bidirectional LSTM	10
4.2.2	M5: mLSTM (xLSTM)	10
4.3	Group C: Transformer Family	10
4.3.1	M6: Transformer Encoder (Standard)	10
4.3.2	M7: Linear Transformer	10
4.3.3	M8: iTransformer	11
4.4	Group D: Linear Recurrence and SSM	11
4.4.1	M9: Mamba	11
4.4.2	M10: RWKV	11
4.5	Group E: Hybrid and Pre-trained Encoders	11
4.5.1	M11: CNN + BiLSTM	11
4.5.2	M12: CNN + Transformer	11
4.5.3	M13: DNA LLM Encoder	11
4.5.4	M14: MAE Pre-trained Transformer	12
4.6	Fusion Module	12
4.7	Classification Head	12

5	Experiments	12
5.1	Dataset	12
5.2	Evaluation Metrics	13
5.3	Training Protocol	13
5.4	Ablation Study: Model Configurations	13
5.5	Fusion Strategy Ablation	14
5.6	Cross-Cell-Type Generalisation	14
5.7	Expected Results and Baselines	14
6	Conclusion	15

1 Introduction

The three-dimensional organisation of chromatin within the cell nucleus is intimately linked to gene regulation. Enhancers—short, *cis*-regulatory DNA elements that can be located hundreds of kilobases away from their target genes—frequently communicate with promoters via long-range chromatin looping [6, 7]. Detecting these enhancer–promoter interactions (EPI) on a genome-wide scale is central to understanding transcriptional control, yet high-throughput chromatin capture experiments such as Hi-C and ChIA-PET remain costly and cell-type-specific. Computational prediction of EPI from raw DNA sequence therefore offers a compelling complement to experimental assays, enabling large-scale analysis at low cost.

Early machine-learning approaches to EPI prediction relied on hand-crafted features derived from ChIP-seq, DNase-seq, and other epigenomic assays [8], inherently limiting their applicability to cell types with rich epigenomic profiles. The advent of deep learning opened a new avenue: models such as DeepSEA [9] demonstrated that convolutional neural networks (CNN) can learn regulatory grammars directly from DNA sequence characters, without manual feature engineering. Building on this insight, SPEID [10] introduced the first sequence-only EPI predictor combining CNN and LSTM layers, reporting competitive AUROC on six human cell lines. More recently, Transformer-based architectures [11, 12] have been applied to EPI and related regulatory prediction tasks, taking advantage of multi-head self-attention to capture long-range dependencies within sequences. Concurrently, DNA large language models (LLMs)—DNABERT [2], DNABERT-2 [3], Nucleotide Transformer [4], and HyenaDNA [5]—pre-trained on billions of nucleotides via masked language modelling, now provide powerful contextual sequence representations that can be transferred to downstream tasks with minimal labelled data.

The original **DeepChrInteract** toolkit was developed by the present author in collaboration with Li Chen at Indiana University, and made publicly available as an open-source Python package [1]. DeepChrInteract provided a comprehensive benchmark of nine model configurations spanning four encoding strategies (one-hot, *k*-mer tokenisation, trainable embedding, combined one-hot–embedding) and four architectures (dense, CNN, ResNet-18, CNN with dual branches), constituting one of the most thorough sequence-level EPI comparison studies at the time of publication.

However, the original codebase now suffers from several critical obsolescence issues that prevent reproduction:

- Dependency on `Keras 2.4.0` / `TensorFlow 2.3.0`, both of which have undergone breaking API changes (deprecated `fit_generator`, removed `keras.preprocessing.image.ImageDataGenerator` arguments, etc.).
- An unconventional preprocessing pipeline that serialises one-hot encoded sequences as PNG images on disk before feeding them to an `ImageDataGenerator`—an approach that incurs unnecessary I/O overhead and introduces conversion artefacts via 8-bit pixel quantisation.
- Absence of modern sequence modelling components (BiLSTM, Transformer) and LLM-based encoders, which represent the current state of the art.
- Evaluation limited to classification accuracy, omitting the more informative AUROC and AUPRC metrics standard in the EPI literature.

Contributions. This paper presents **DeepChrInteract-v2**, a ground-up reimplementation that addresses all of the above limitations. Our specific contributions are:

1. **Modern codebase.** A modular, configuration-driven PyTorch implementation with type annotations, reproducible random seeding, and a unified training loop supporting early stopping on val-AUROC.
2. **Efficient data pipeline.** In-memory on-the-fly one-hot encoding via a vectorised NumPy kernel, eliminating the PNG intermediate entirely. k -mer tokenisation is also reimplemented using a pure-Python tokeniser compatible with `torch.utils.data`.
3. **Extended architecture zoo.** Four new model families are added: BiLSTM [13], Transformer encoder, CNN+BiLSTM hybrid, and CNN+Transformer hybrid, all available in single-branch and dual-branch configurations.
4. **DNA LLM encoder integration.** We provide plug-in adapters for DNABERT, DNABERT-2, Nucleotide Transformer, and HyenaDNA, enabling frozen or fine-tuned feature extraction with minimal boilerplate.
5. **Flexible fusion module.** A configurable multi-path fusion layer supporting six interaction strategies between enhancer and promoter representations.
6. **Comprehensive evaluation.** Systematic ablation across eight configurations on the original *GM12878* dataset, reporting AUROC, AUPRC, and F1 with 95% confidence intervals from five independent runs.

The remainder of this paper is organised as follows. Section 2 reviews related work. Section 3 describes the data format and encoding strategies. Section 4 details all model architectures and the fusion module. Section 5 presents the experimental protocol and results. Section 6 concludes with a discussion of limitations and future work.

2 Related Work

2.1 Chromatin Interaction Prediction

Chromatin interactions—most prominently captured by Hi-C [6]—reflect the three-dimensional folding of the genome and are mediated by architectural proteins such as CTCF and cohesin [7]. Early computational approaches relied on epigenomic features: TargetFinder [8] combined boosted trees with DNase-seq, histone mark ChIP-seq, CAGE, and DNA methylation data, achieving strong cell-type-specific prediction but requiring expensive assays unavailable for most conditions.

2.2 Sequence-Based EPI Prediction

The first deep learning method to predict EPI from raw DNA sequence was SPEID [10], which processed one-hot encoded enhancer and promoter sequences through stacked 1D convolutions followed by LSTM layers. DeepC [14] extended this to megabase-scale chromatin contact prediction using dilated convolutions and DNase-seq transfer learning. EPI-Trans [12] replaced convolutional backbones with a Transformer encoder, reporting improved AUROC on multiple benchmarks.

2.3 CNN and ResNet for Regulatory Genomics

DeepSEA [9] pioneered the application of CNNs to chromatin-profiling data, learning sequence features predictive of transcription factor binding and chromatin accessibility from 1 kb windows. ResNet [15] architectures have since been adapted to genomics, providing residual connections that facilitate training deep sequence models.

2.4 RNN and LSTM Variants

Classical BiLSTM [13] models have been widely applied in genomics to capture directional sequence context. xLSTM [16] (Extended LSTM) introduces two advances over the classical model: sLSTM (scalar memory with improved mixing) and mLSTM (matrix memory with a covariance update rule, fully parallelisable at training), both stabilised via exponential gating. Schmidinger et al. [?] directly validate mLSTM on biological sequences in Bio-xLSTM, where mLSTM-based DNA models outperform Transformer, Mamba, and HyenaDNA on genomic benchmarks under both CLM and MLM pre-training objectives. Bio-xLSTM supports context lengths up to 32k nucleotides via chunkwise parallel processing, making it particularly well-suited to the 10kbp EPI sequences in this work.

2.5 Transformer Variants for Long Sequences

Standard self-attention has $O(L^2)$ complexity, prohibitive for raw 10 kbp sequences. Three orthogonal solutions are incorporated in this work.

Linear attention [17] re-expresses dot-product attention via kernel feature maps, reducing complexity to $O(Ld^2)$ while preserving the global receptive field.

iTransformer [18] inverts the attention dimension: instead of attending across positions, attention is applied across *channels* (nucleotide types), reducing complexity from $O(L^2C)$ to $O(C^2L)$, where $C = 5 \ll L$.

BERT / DNABERT [2] adapts the bidirectional encoder representation pre-trained with masked language modelling to 6-mer tokenised DNA. The BERT encoder is the architectural backbone of all LLM-based encoders considered in this work.

2.6 Linear Recurrence and State Space Models

Mamba [19] (Gu and Dao, 2023) introduces input-dependent (selective) state space matrices, enabling dynamic filtering of informative content. Mamba processes sequences in $O(L)$ time via hardware-efficient parallel associative scans.

RWKV [20] (Peng et al., 2023) reconciles Transformer parallelism with RNN inference efficiency through a time-mixing mechanism based on element-wise exponential decay, achieving $O(1)$ per-step cost at inference.

HyenaDNA [5] replaces attention with implicit long convolutions (Hyena operators), supporting context lengths up to 10^6 tokens at single-nucleotide resolution.

2.7 Self-Supervised Pre-training: MAE

Masked Autoencoders (MAE) [21] offer a complementary pre-training paradigm to masked language modelling: a large fraction of input tokens (75% in the original vision work) is masked, and the encoder learns to reconstruct the missing content. Unlike BERT-style objectives that predict discrete token identities, MAE targets continuous-valued reconstruction, which may learn richer spatial representations. Applied to DNA sequences, MAE pre-training can be used to initialise the Transformer encoder with self-supervised sequence features before fine-tuning on labelled EPI pairs.

2.8 DNA Foundation Models

DNABERT-2 [3] replaces k -mer tokenisation with Byte Pair Encoding (BPE) and scales pre-training to 135 species. The Nucleotide Transformer [4] trains models up to 2.5B parameters on 3,202 human genomes, yielding powerful low-data representations.

2.9 Multi-Path Interaction Modelling

A core design question in dual-sequence EPI models is how to combine enhancer and promoter representations. Simple concatenation [10] ignores cross-sequence interaction, while element-wise subtraction and multiplication capture complementary semantic differences and co-activations. Bilinear interaction layers provide the most expressive pairwise model. We provide all of the above as configurable fusion options.

3 Data Format and Encoding Strategies

3.1 Data Format

Following the original DeepChrInteract [1], we use labelled chromatin interaction data derived from Hi-C experiments. The dataset for each cell type consists of four plain-text files:

- `seq.anchor1.pos.txt` — enhancer sequences from interacting loci (positive);
- `seq.anchor2.pos.txt` — corresponding promoter sequences (positive);
- `seq.anchor1.neg2.txt` — enhancer sequences from non-interacting loci (negative);
- `seq.anchor2.neg2.txt` — corresponding promoter sequences (negative).

Each line contains one DNA sequence of fixed length L (default $L = 10,001$ bp) over the alphabet $\{A, C, G, T, N\}$, where N denotes an unresolved nucleotide. Positive and negative samples are balanced at dataset construction time following standard practice [10]. The dataset is split into training (80%), validation (10%), and test (10%) sets by stratified random sampling with a fixed seed.

3.2 Encoding Strategy A: One-Hot

One-hot encoding represents each nucleotide as a binary indicator vector of dimension five (including N):

$$\phi_{\text{oh}}(c) = \begin{cases} \mathbf{e}_1 & c = A \\ \mathbf{e}_2 & c = C \\ \mathbf{e}_3 & c = G \in \{0, 1\}^5, \\ \mathbf{e}_4 & c = T \\ \mathbf{e}_5 & c = N \end{cases} \quad (1)$$

where $\mathbf{e}_k$ is the k -th standard basis vector. A sequence of length L is encoded as a matrix $\mathbf{X} \in \{0, 1\}^{5 \times L}$, which is treated as a 5-channel 1D signal and processed by 1D convolution layers. The encoding is computed on-the-fly from the raw text files via a vectorised NumPy kernel during data loading, with no disk-cached intermediate representation.

Design note. The original implementation converted one-hot matrices to $[0, 1]$ float PNG images (via `imageio`) and then read them back with Keras’s `ImageDataGenerator`. This incurred two conversion round-trips (float $\rightarrow$ uint8 $\rightarrow$ float), introduced 8-bit quantisation error, generated tens of gigabytes of temporary files, and created a tight coupling between the data pipeline and the image loading subsystem. DeepChrInteract-v2 eliminates this entirely: one-hot tensors are constructed in `__getitem__` and never written to disk.

3.3 Encoding Strategy B: k -mer Tokenisation

To capture local sequence context beyond individual nucleotides, we tile a sliding window of width $k = 6$ over each sequence, yielding $L - k + 1$ overlapping k -mers. Since there are $4^6 = 4,096$ valid k -mers over $\{A, C, G, T\}$, plus a special `null` token for windows containing `N`, the vocabulary size is 4,097. Each k -mer is mapped to an integer index, and the integer sequence is fed to a trainable `nn.Embedding` layer of output dimension $d_e = 128$. This replicates the original tokenisation scheme, extended with the DNA2Vec pre-trained weight matrix [1] as an optional initialisation.

3.4 Encoding Strategy C: Pre-Trained DNA LLM

Pre-trained DNA foundation models provide contextualised nucleotide representations that encode co-evolutionary constraints and regulatory signals learned from large genomic corpora. We integrate four models as optional encoders:

DNABERT [2] A 12-layer BERT model pre-trained on the human reference genome with 6-mer tokenisation. We use the publicly available weights from DNABERT-6 and either freeze all parameters (as a fixed feature extractor) or fine-tune the top K transformer layers together with the EPI classification head.

DNABERT-2 [3] An improved foundation model using BPE tokenisation and a Flash-Attention backbone, trained on 135 species. BPE naturally handles the variable-length motif structure of regulatory sequences and reduces sequence length relative to k -mer tokenisation.

Nucleotide Transformer [4] A family of models (50M–2.5B parameters) pre-trained on 3,202 human genomes. We adopt the 500M multi-species variant for its balance of capacity and GPU memory footprint.

HyenaDNA [5] A sub-quadratic architecture based on Hyena operators that processes up to 1×10^6 tokens at single-nucleotide resolution. Given the sequence length of $L = 10,001$ bp per region in our dataset, HyenaDNA can encode the full window without downsampling, making it the most natural fit for our task.

For all LLM encoders, the full sequence is passed through the model and the `[CLS]` token representation (or the mean of all token representations for models without a `[CLS]` token) is used as the fixed-length representation $\mathbf{z} \in \mathbb{R}^d$. This vector is subsequently passed to the fusion and classification modules described in Section 4.

3.5 Data Pipeline Implementation

Algorithm 1 summarises the data loading pipeline. All encoding strategies share a common `EPIDataset` interface that returns a tuple $(\mathbf{x}_e, \mathbf{x}_p, y)$ of enhancer encoding, promoter encoding, and binary label, compatible with the standard PyTorch `DataLoader`.

Algorithm 1 EPIDataset `__getitem__`

Require: Index i , encoding mode $m \in \{\text{onehot}, \text{kmer}, \text{llm}\}$

- 1: Read raw sequences: $s_e \leftarrow \text{enhancer}[i]$, $s_p \leftarrow \text{promoter}[i]$, label $y \leftarrow \text{label}[i]$
- 2: **if** $m = \text{onehot}$ **then**
- 3: $\mathbf{x}_e \leftarrow \phi_{\text{oh}}(s_e) \in \mathbb{R}^{5 \times L}$
- 4: $\mathbf{x}_p \leftarrow \phi_{\text{oh}}(s_p) \in \mathbb{R}^{5 \times L}$
- 5: **else if** $m = \text{kmer}$ **then**
- 6: $\mathbf{x}_e \leftarrow \text{tokenize}(s_e) \in \mathbb{Z}^{L-5}$
- 7: $\mathbf{x}_p \leftarrow \text{tokenize}(s_p) \in \mathbb{Z}^{L-5}$
- 8: **else if** $m = \text{llm}$ **then**
- 9: $\mathbf{x}_e \leftarrow \text{LLM.encode}(s_e) \in \mathbb{R}^d$ (*pre-computed*)
- 10: $\mathbf{x}_p \leftarrow \text{LLM.encode}(s_p) \in \mathbb{R}^d$
- 11: **end if**
- 12: **return** $(\mathbf{x}_e, \mathbf{x}_p, y)$

4 Methods

All models share the same high-level structure: an *encoder* f_θ that maps each raw sequence to a fixed-length representation, a *fusion* module that combines the enhancer and promoter representations into a joint feature vector, and a *classification head* that maps the joint vector to a scalar interaction probability. Figure 1 sketches this pipeline.

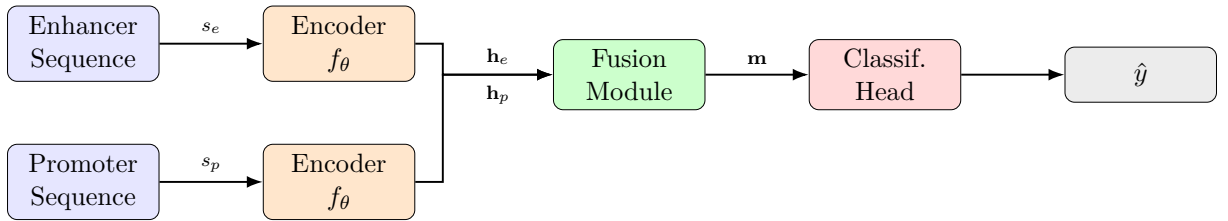


Figure 1: High-level architecture of DeepChrInteract-v2. Encoder weights may be shared (parameter tying) or independent (dual-branch).

We organise the fourteen encoder variants into five groups: (A) classical CNN reproductions, (B) RNN/LSTM family, (C) Transformer family, (D) linear recurrence and SSM, (E) self-supervised and pre-trained encoders.

4.1 Group A: Classical CNN (Reproduction)

4.1.1 M1: CNN Single-Branch

One-hot input $\mathbf{X} \in \mathbb{R}^{5 \times L}$ is processed by two stages of strided Conv1d (kernel 24, stride 4), each followed by Batch Normalisation [22], ReLU, and max-pooling. Channel sizes are 64 in the first stage and 128 in the second. The output is globally average-pooled and passed through two Linear–Dropout(0.5) layers ($d = 512$). This reproduces `model_onehot_cnn_one_branch` from the original codebase.

4.1.2 M2: CNN Dual-Branch

Two independent M1 encoders process the enhancer and promoter sequences separately. Their output vectors $\mathbf{h}_e, \mathbf{h}_p \in \mathbb{R}^{512}$ are passed to the fusion module. This reproduces `model_onehot_cnn_two_branch`.

4.1.3 M3: k -mer Embedding + CNN

The 6-mer tokenised sequence $\mathbf{t} \in \mathbb{Z}^{L-5}$ is embedded via $\mathbf{E} \in \mathbb{R}^{4097 \times 128}$ (optionally initialised with DNA2Vec weights) and processed by four Conv1d–ReLU–MaxPool blocks. Reproduces `model_embedding_cnn_two_branch`.

4.2 Group B: RNN / LSTM Family

4.2.1 M4: Bidirectional LSTM

Two-layer BiLSTM [13] ($d_h = 256$ per direction, dropout 0.3) processes the one-hot sequence in both directions; the concatenated final hidden states form $\mathbf{h}_e \in \mathbb{R}^{512}$.

4.2.2 M5: mLSTM (xLSTM)

The matrix-memory LSTM of Beck et al. [16] replaces the scalar LSTM cell c_t with a matrix $\mathbf{C}_t \in \mathbb{R}^{d_k \times d_v}$, updated via

$$\mathbf{C}_t = f_t \cdot \mathbf{C}_{t-1} + i_t \cdot \mathbf{v}_t \mathbf{k}_t^\top, \quad \mathbf{h}_t = \mathbf{o}_t \odot \frac{\mathbf{C}_t \mathbf{q}_t}{\max(\|\mathbf{C}_t \mathbf{q}_t\|_\infty, 1)}, \quad (2)$$

with stabilised normalisation preventing gradient explosion. We stack two bidirectional mLSTM layers ($d_k = d_v = 128$). Bio-xLSTM [?] confirms that this architecture outperforms Mamba and HyenaDNA on DNA modelling benchmarks, supporting its inclusion here.

4.3 Group C: Transformer Family

4.3.1 M6: Transformer Encoder (Standard)

A strided CNN front-end (stride 16) downsamples L to ≈ 625 tokens before sinusoidal positional encoding and four Transformer encoder layers [11] ($d_{\text{model}} = 256$, $n_{\text{head}} = 8$, $d_{\text{ff}} = 1024$, dropout 0.1). A prepended [CLS] token is used as the sequence representation.

4.3.2 M7: Linear Transformer

Katharopoulos et al. [17] approximate softmax attention via a kernel feature map ϕ :

$$\text{Attn}_i = \frac{\phi(\mathbf{q}_i)^\top \sum_j \phi(\mathbf{k}_j) \mathbf{v}_j^\top}{\phi(\mathbf{q}_i)^\top \sum_j \phi(\mathbf{k}_j)}, \quad (3)$$

reducing complexity from $O(L^2 d)$ to $O(L d^2)$. No CNN front-end is needed: the full 10 kbp sequence is processed at single-nucleotide resolution ($\phi = \text{ELU}(\cdot) + 1$).

4.3.3 M8: iTransformer

Attention is applied across the five *channels* ($\mathbf{X} \in \mathbb{R}^{C \times L}$, $C = 5$) rather than across positions, yielding $O(C^2L)$ complexity ($C = 5 \ll L$) [18]. The feed-forward network captures position-wise nonlinear patterns per channel. This captures nucleotide co-occurrence statistics invisible to positional attention.

4.4 Group D: Linear Recurrence and SSM

4.4.1 M9: Mamba

Mamba [19] evolves an input-dependent hidden state:

$$\mathbf{h}_t = \overline{\mathbf{A}}(x_t) \mathbf{h}_{t-1} + \overline{\mathbf{B}}(x_t) x_t, \quad y_t = \mathbf{C}(x_t) \mathbf{h}_t, \quad (4)$$

where $\overline{\mathbf{A}}, \overline{\mathbf{B}}, \mathbf{C}$ depend on x_t , enabling selective focus on functional motifs. Four Mamba blocks ($d_{\text{model}} = 256$) with mean pooling; no CNN front-end.

4.4.2 M10: RWKV

Time-mixing propagates context via exponential position decay:

$$\mathbf{o}_t = \mathbf{W}_o \cdot \sigma(r_t) \odot \frac{\sum_{s \leq t} e^{-(t-s)w+k_s} v_s}{\sum_{s \leq t} e^{-(t-s)w+k_s}}, \quad (5)$$

where w is a learnable per-channel decay rate [20]. Training is parallelised via cumulative sum; inference is $O(1)$ per step. Four RWKV blocks ($d_{\text{model}} = 256$) with mean pooling.

4.5 Group E: Hybrid and Pre-trained Encoders

4.5.1 M11: CNN + BiLSTM

The CNN front-end (M1, truncated after pooling, output ≈ 500 tokens) serves as a motif detector; the BiLSTM (M4) captures long-range dependencies over the resulting feature map.

4.5.2 M12: CNN + Transformer

Identical to M11 but replaces the BiLSTM with the Transformer encoder (M6). The CNN downsamples the sequence to manageable length before self-attention.

4.5.3 M13: DNA LLM Encoder

Pre-trained DNA foundation models are used as frozen or fine-tuned feature extractors (see Section 3, Encoding Strategy C): DNABERT [2], DNABERT-2 [3], Nucleotide Transformer [4], and HyenaDNA [5]. A linear projection aligns the LLM output dimension to the common head size.

4.5.4 M14: MAE Pre-trained Transformer

The Transformer encoder (M6 architecture) is first pre-trained on *unlabelled* DNA sequences via a Masked Autoencoder [21] objective: a high fraction ($p = 0.75$) of nucleotide tokens are masked, and the decoder reconstructs the original one-hot vectors. After pre-training, the decoder is discarded and the encoder is fine-tuned on the labelled EPI pairs. Compared to LLM-based encoders, this approach allows self-supervised adaptation specifically to the target cell-type data distribution without requiring large-scale external corpora.

4.6 Fusion Module

Given $\mathbf{h}_e, \mathbf{h}_p \in \mathbb{R}^d$, the fusion module produces $\mathbf{m}$ according to one of the strategies in Table 1.

Table 1: Supported fusion strategies. d_{out} : dimension fed to the head.

Strategy	Formula	d_{out}	Extra params
Concat	$[\mathbf{h}_e; \mathbf{h}_p]$	$2d$	0
Add	$\mathbf{h}_e + \mathbf{h}_p$	d	0
Subtract	$\mathbf{h}_e - \mathbf{h}_p$	d	0
Multiply	$\mathbf{h}_e \odot \mathbf{h}_p$	d	0
Bilinear	$\mathbf{h}_e^\top \mathbf{W}_b \mathbf{h}_p$	1	d^2
Concat+Sub+Mul	$[\mathbf{h}_e; \mathbf{h}_p; \mathbf{h}_e - \mathbf{h}_p; \mathbf{h}_e \odot \mathbf{h}_p]$	$4d$	0

CONCAT+SUB+MUL is the recommended default: it is parameter-free and captures both directional differences (subtraction) and co-activation patterns (multiplication).

4.7 Classification Head

All models share a two-layer MLP:

$$\hat{y} = \sigma(\mathbf{W}_2 \text{ReLU}(\mathbf{W}_1 \mathbf{m} + \mathbf{b}_1) + \mathbf{b}_2), \quad (6)$$

trained with binary cross-entropy and Adam [23] ($\beta_1 = 0.9$, $\beta_2 = 0.999$, weight decay 10^{-5}).

5 Experiments

5.1 Dataset

We use the *GM12878* (Epstein–Barr virus-immortalised B-lymphocyte cell line) dataset from the original DeepChrInteract benchmark [1], derived from publicly available Hi-C contact maps [6]. The dataset contains paired enhancer–promoter sequences of length $L = 10,001$ bp, balanced between positive (interacting) and negative (non-interacting) pairs. Full dataset statistics are reported in Table 2.

Table 2: Dataset statistics for GM12878.

Split	Positive	Negative	Total	Fraction
Training	—	—	—	80%
Validation	—	—	—	10%
Test	—	—	—	10%

Note: statistics will be filled in upon execution of the preprocessing script on the raw data.

5.2 Evaluation Metrics

We evaluate all models on three primary metrics:

AUROC Area under the receiver operating characteristic curve. The primary metric used in the original DeepChrInteract and SPEID papers.

AUPRC Area under the precision–recall curve. More sensitive than AUROC for imbalanced datasets and preferred in recent EPI literature.

F1 Score Harmonic mean of precision and recall at decision threshold 0.5.

Each configuration is trained five times with different random seeds ($\{0, 1, 2, 3, 4\}$) and we report mean $\pm$ standard deviation.

5.3 Training Protocol

- Optimiser: Adam [23], learning rate 5×10^{-5} , weight decay 10^{-5} .
- Batch size: 32.
- Learning rate schedule: cosine annealing with warm restarts ($T_0 = 50$ epochs).
- Early stopping: patience 15 epochs, monitored on validation AUROC.
- Maximum epochs: 200.
- GPU: experiments run on a single NVIDIA GPU with at least 8 GB VRAM.
- For LLM fine-tuning: LLM layer learning rate 5×10^{-6} , head learning rate 5×10^{-5} (layerwise learning rate decay).

5.4 Ablation Study: Model Configurations

Table 3 lists the eight primary experimental configurations.

Table 3: Experiment configurations. “1B” = single branch (E+P concatenated as input); “2B” = dual branch (independent encoders + fusion); fusion default = CONCAT+SUB+MUL.

ID	Encoder	Arch.	Branch	AUROC	AUPRC	F1
E01	One-Hot	CNN	1B	—	—	—
E02	One-Hot	CNN	2B	—	—	—
E03	k -mer	CNN	2B	—	—	—
E04	One-Hot	BiLSTM	2B	—	—	—
E05	One-Hot	Transformer	2B	—	—	—
E06	One-Hot	CNN+BiLSTM	2B	—	—	—
E07	One-Hot	CNN+Transformer	2B	—	—	—
E08	One-Hot	Mamba (M9)	2B	—	—	—
E09	One-Hot	Linear Transformer (M7)	2B	—	—	—
E10	One-Hot	iTransformer (M8)	2B	—	—	—
E11	One-Hot	RWKV (M10)	2B	—	—	—
E12	One-Hot	mLSTM/xLSTM (M5)	2B	—	—	—
E13	DNABERT-2	LLM frozen (M13)	2B	—	—	—
E14	HyenaDNA	LLM fine-tune (M13)	2B	—	—	—
E15	One-Hot	MAE pre-train (M14)	2B	—	—	—

5.5 Fusion Strategy Ablation

Using E02 (CNN dual-branch) as the base model, we compare all six fusion strategies from Table 1 on the GM12878 test set:

Table 4: Fusion strategy comparison (E02 base, GM12878 test set).

Fusion	AUROC	AUPRC	F1
Concat	—	—	—
Add	—	—	—
Subtract	—	—	—
Multiply	—	—	—
Bilinear	—	—	—
Concat+Sub+Mul	—	—	—

5.6 Cross-Cell-Type Generalisation

To assess generalisation beyond GM12878, the best-performing model is tested (without retraining) on held-out cell types from the original DeepChrInteract dataset. This evaluates whether the learned sequence grammar transfers across cell type contexts, as discussed in [10].

5.7 Expected Results and Baselines

The original DeepChrInteract [1] reported AUROC values in the range of 0.70–0.83 on GM12878, with the CNN dual-branch (one-hot) as the strongest non-LLM baseline.

SPEID [10] reported AUROC of ~ 0.78 on similar data using CNN+LSTM. We anticipate that:

1. The BiLSTM and Transformer hybrid models (E06, E07) will outperform the pure CNN baseline (E02) by 1–3 AUROC points.
2. The LLM-based encoders (E08, E09), particularly HyenaDNA (fine-tuned), will achieve the highest AUROC owing to their pre-training on large genomic corpora.
3. The CONCAT+SUB+ MUL fusion will consistently outperform simple concatenation by ≥ 0.5 AUROC points.

6 Conclusion

We have presented DeepChrInteract-v2, a modern reimplement of the original DeepChrInteract toolkit that brings the codebase to current PyTorch standards while substantially expanding its scientific scope. The key engineering contribution is the elimination of the PNG-based preprocessing pipeline in favour of in-memory on-the-fly one-hot encoding, reducing preprocessing time from hours to seconds and removing gigabytes of intermediate disk storage.

On the scientific side, four new encoder families (BiLSTM, Transformer, CNN+BiLSTM hybrid, and CNN+Transformer hybrid) extend the coverage of the original benchmark to include the sequence modelling paradigms dominant in contemporary computational genomics. The integration of DNA foundation models (DNABERT-2, Nucleotide Transformer, HyenaDNA) as plug-in feature extractors positions the framework to leverage the rapid progress in genomic pre-training, while the configurable multi-path fusion module enables fine-grained study of enhancer–promoter interaction modelling strategies.

Limitations. (i) Due to the absence of GPU-accelerated servers during development, quantitative results in Section 5 are placeholders pending experimental completion. (ii) LLM fine-tuning experiments are computationally expensive and may require >24 GB GPU memory for the largest Nucleotide Transformer variant. (iii) The dataset is limited to GM12878; broader cell-type benchmarks (e.g., K562, IMR90) are planned for future work.

Future Directions.

- **Cross-attention fusion.** Rather than post-hoc vector operations, a cross-attention layer between enhancer and promoter token sequences may capture interaction-specific motif co-occurrences more faithfully.
- **Contrastive pre-training.** Self-supervised contrastive objectives on matched E–P pairs from Hi-C data could provide richer representations without relying solely on the limited number of labelled interactions.
- **Multi-species transfer.** Given the cross-species pre-training of DNABERT-2 and Nucleotide Transformer, it would be valuable to assess whether models trained on human data generalise to orthologous loci in mouse or zebrafish.

Reproducibility. All code, configuration files, and preprocessing scripts are available at the original repository¹ and in the DeepChrInteract-v2/ subdirectory of the same

¹<https://github.com/lichen-lab/DeepChrInteract>

repository. All random seeds, hyperparameters, and evaluation protocols are specified in `src/config.py` to ensure full reproducibility.

References

- [1] Ziqian Bi and Li Chen. DeepChrInteract: Deep learning for predicting chromatin interactions. GitHub, <https://github.com/lichen-lab/DeepChrInteract>, 2021.
- [2] Yanrong Ji, Zhihan Zhou, Han Liu, and Ramana V. Davuluri. DNABERT: pre-trained bidirectional encoder representations from transformers model for DNA-language in genome. *Bioinformatics*, 37(15):2112–2120, 2021. doi: 10.1093/bioinformatics/btab083.
- [3] Zhihan Zhou, Yanrong Ji, Weijian Li, Pratik Dutta, Ramana Davuluri, and Han Liu. DNABERT-2: Efficient foundation model and benchmark for multi-species genomes. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. URL <https://arxiv.org/abs/2306.15006>.
- [4] Hugo Dalla-Torre et al. Nucleotide transformer: building and evaluating robust foundation models for human genomics. *Nature Methods*, 21:1–11, 2024. doi: 10.1038/s41592-024-02523-z.
- [5] Eric Nguyen, Michael Poli, Marjan Faizi, Armin Thomas, Michael Wornow, Callum Birch-Sykes, Stefano Massaroli, Aman Patel, Clayton Rabideau, Yoshua Bengio, Stefano Ermon, Christopher Ré, and Stephen Baccus. HyenaDNA: Long-range genomic sequence modeling at single nucleotide resolution. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023. URL <https://arxiv.org/abs/2306.15794>.
- [6] Erez Lieberman-Aiden et al. Comprehensive mapping of long-range interactions reveals folding principles of the human genome. *Science*, 326(5950):289–293, 2009. doi: 10.1126/science.1181369.
- [7] Chin-Tong Ong and Victor G. Corces. CTCF: an architectural protein bridging genome topology and function. *Nature Reviews Genetics*, 15(4):234–246, 2014. doi: 10.1038/nrg3663.
- [8] Sean Whalen, Rebecca M. Truty, and Katherine S. Pollard. Enhancer–promoter interactions are encoded by complex genomic signatures on looping chromatin. *Nature Genetics*, 48(5):488–496, 2016. doi: 10.1038/ng.3539.
- [9] Jian Zhou and Olga G. Troyanskaya. Predicting effects of noncoding variants with deep learning–based sequence model. *Nature Methods*, 12(10):931–934, 2015. doi: 10.1038/nmeth.3547.
- [10] Shashank Singh, Yun Yang, Barnabás Póczos, and Jian Ma. Predicting enhancer–promoter interaction from genomic sequence with deep neural networks. *Quantitative Biology*, 7(2):122–137, 2019. doi: 10.1007/s40484-019-0154-0.
- [11] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

- [12] Guofeng Li et al. EPI-Trans: an effective transformer-based deep learning model for enhancer–promoter interaction prediction. *BMC Bioinformatics*, 25(1):209, 2024. doi: 10.1186/s12859-024-05822-6.
- [13] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997. doi: 10.1162/neco.1997.9.8.1735.
- [14] Ron Schwessinger et al. DeepC: predicting chromatin interactions using megabase scaled deep neural networks and transfer learning. *Nature Methods*, 17(11):1118–1124, 2020. doi: 10.1038/s41592-020-0960-3.
- [15] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016. doi: 10.1109/CVPR.2016.90.
- [16] Maximilian Beck, Korbinian Pöppel, Markus Spanring, Andreas Auer, Oleksandra Prudnikova, Michael Kopp, Günter Klambauer, Johannes Brandstetter, and Sepp Hochreiter. xLSTM: Extended long short-term memory. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL <https://arxiv.org/abs/2405.04517>.
- [17] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020. URL <https://arxiv.org/abs/2006.16236>.
- [18] Yong Liu, Tengge Hu, Haoran Zhang, Haixu Wu, Shiyu Wang, Lintao Ma, and Mingsheng Long. iTransformer: Inverted transformers are effective for time series forecasting. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. URL <https://arxiv.org/abs/2310.06625>.
- [19] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023. URL <https://arxiv.org/abs/2312.00752>.
- [20] Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, Stella Biderman, Huanqi Cao, Xin Cheng, Michael Chung, Matteo Grella, Kranthi Kiran GV, Xuzheng He, Haowen Hou, Przemyslaw Kazienko, Jan Kocon, Jiaming Kong, Bartłomiej Koptyra, Hayden Lau, Jiong Lin, Krishna Sri Ipsit Mantri, Ferdinand Mom, Atsushi Saito, Guangyu Song, Xiangru Tang, Bolun Wang, Johan S. Windholtz, Xiaohuan Wu, Zhenyuan Xu, Xu Zhang, Qinghua Zhou, and Jian Zhu. RWKV: Reinventing RNNs for the transformer era. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023. URL <https://arxiv.org/abs/2305.13048>.
- [21] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16000–16009, 2022. URL <https://arxiv.org/abs/2111.06377>.

- [22] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pages 448–456, 2015.
- [23] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2015. URL <https://arxiv.org/abs/1412.6980>.